

Hi, this is Monica Rajagopal, from Team MARS, presenting Decision Packet Automation for Life Underwriting and Claims.

The scope of this proof of concept is limited to underwriting only.

Life insurers receive large volumes of unstructured information, including medical reports, APS documents, lab results, applications, policy records, claim forms, death certificates, and case notes. Today, underwriters and claims examiners often spend significant time reading documents, locating key evidence, creating summaries, checking guidelines, and documenting their decisions.

Our solution uses Claude AI, document intelligence, OCR, and a configurable business rules engine to automate the preparation of a complete, evidence-grounded Decision Packet.

The process begins when documents and case data are received. OCR converts scanned files into usable text, and Claude extracts relevant information such as diagnoses, medications, lab results, dates, policy details, claim details, and other risk indicators. From there, each case moves through four stages: ingest and extract, normalise and cite, validate, and decide.

The solution then normalises this information into a structured evidence model. Every key finding is linked back to its source document, page number, date, value, and confidence score. This creates a transparent medical or claims timeline that the reviewer can quickly validate.

Next, the tool checks whether the evidence is complete and consistent. If information is missing, conflicting, outdated, or uncertain, the system does not make an unsupported recommendation. Instead, it creates a targeted request for the exact evidence needed, and routes the case for review.

For cases with sufficient evidence, the solution retrieves carrier-approved underwriting guidelines and applies them through a controlled rules engine. Claude then creates a Decision Packet containing a case summary, relevant findings, source citations, triggered rules, required actions, and a recommended disposition.

The underwriter or claims examiner remains in control. They can approve, modify, override, request additional information, or escalate the case to a senior underwriter, a medical reviewer, the special investigations unit, or legal.

This solution helps insurers reduce manual document review, improve decision consistency, accelerate turnaround time, and create audit-ready documentation. It can begin as a focused proof of value, and scale across underwriting and claims workflows through integrations with policy, claims, document management, and data platforms.

Decision Packet Automation turns fragmented insurance documents into a clear, traceable, and reviewer-ready decision experience.

That is the solution in outline. Now let me show you the working prototype.

On screen is a single term life case, exactly as it arrives. Six documents: the application, an attending physician statement, a lab report, a motor vehicle record, the prior underwriting history, and an email from the advisor. In production these are produced by OCR from scanned PDFs, faxes and images. Here they have already been converted to text, which is where our pipeline picks up.

Notice that the formats are mixed: plain text, a spreadsheet, a PDF and a Word document. That is normal for a real case file, and the pipeline handles all of them.

Now we run the pipeline for this case with a single command. While it runs, here is what happens behind the scenes, in four stages.

First, ingestion opens every document, pulls out the text, and tags each chunk with the case and document type. No judgement is applied here, and it finishes almost instantly.

Then Claude reads that tagged text and extracts the underwriting facts, notes anything missing or any risk signals, and ties every fact back to its exact source. This is usually the longest of the four stages.

Next, Claude drafts the reviewer-facing narrative: a summary, a set of concrete next steps, and a draft recommendation.

Finally, Claude maps the findings onto our underwriting framework and chooses a band for each of its nineteen factors. All the arithmetic after that — multiplying, summing, and matching to a risk band — is done in plain Python, not by the model, so the same findings always produce the same score.

And there it is — the packet is now assembled.

On disk we have four files for this case. The tagged text from stage one. The extracted facts with their citations. The risk score with its factor breakdown. And the final packet, which combines all of it.

This is the reviewer's view. Each case shows its risk band, its score, and the suggested action, colour coded so a queue can be triaged at a glance. Across these cases the bands run the full range, from Low through to Very High.

Opening a case gives us the full packet.

At the top are the headline numbers: the risk score, the risk band, and the suggested underwriting action that the framework attaches to that band.

Below that is the summary. This is the brief a reviewer reads first.

Then the key fields, numbered so they are easy to refer to. Every value carries the document it came from and the exact quoted words that support it, so the reviewer can confirm any fact without reopening the source file.

Missing information lists what a reviewer would expect to see but cannot find in the file, and why it matters.

Risk flags are categorised and rated by severity, each with its own citation.

The score breakdown shows all nineteen factors: the band chosen, the weight, and the contribution to the total. Where the file contains no evidence for a factor, it is marked as not addressed and scored favourably, rather than the system guessing.

Then the suggested action path, which gives concrete next steps in priority order, and the draft rationale behind the recommendation.

Finally the governance block records the review status and confirms that every citation in the packet resolves back to a real source document.

Every packet is a draft prepared for an underwriter. It is not an approval, a decline, or a binding decision. The system assembles and cites the evidence. The underwriter makes the call.